Diário de Experimentos

Sprint 3 — Implementação, parte 1 · Armazenamento, permissões e função de gravação

Equipe Thomas Alane, Melina Pissolato, Davi Araujo e Eduardo Palhares Reale Pereira
Repositório https://github.com/thomasalane/Projeto-Cloud
Ambiente LocalStack (emulador local da AWS), sem conta AWS real e sem custo

---

Nesta sprint o desenho da Sprint 2 saiu do papel. Tudo foi criado por script, no LocalStack: o bucket dos gráficos, a tabela dos experimentos, o papel do IAM e a função que grava, com os registros no CloudWatch Logs e um `deploy.sh` que reconstrói tudo do zero.

Commit da sprint: `c624bc9 sprint 3` (e `b5a4b83`, que marca os scripts como executáveis para Mac e Linux).

1. O que a sprint entrega

Entrega pedida no roteiro	Onde está	Requisito
Bucket S3 criado por script	`infra/01-bucket.sh` → bucket `diario-graficos`	RF-02
Tabela DynamoDB com a chave justificada na Sprint 2	`infra/02-tabela.sh` → tabela `diario-experimentos`, partição `projeto`, ordenação `id_experimento`	RF-01, RF-03, RNF-01
Papel IAM com política anexada	`infra/03-papel.sh` → papel `diario-papel-gravar`	RNF-02, RD-02
Função Lambda gravando no DynamoDB e no S3	`infra/04-funcao-gravar.sh` e `app/gravar/handler.py` → função `diario-gravar`	RF-01, RF-02, RF-04, RNF-03
Registros de execução no CloudWatch Logs	grupo `/aws/lambda/diario-gravar`, uma linha por gravação, recusa ou falha	RNF-04
`deploy.sh` que recria tudo do zero	`infra/deploy.sh`	regra 1 do projeto
`destroy.sh` que remove tudo	`infra/destroy.sh`	regra 1 do projeto
Cargas de teste	`eventos/gravar-ok.json` (US-01, cenário 1) e `eventos/gravar-valor-baixo.json` (US-01, cenário 2)	US-01

Todos os nomes de recurso estão em um lugar só, o `infra/config.sh`, com o prefixo da equipe, `diario`. É o único arquivo que muda para levar o projeto à AWS real.

2. Estrutura de pastas

```
infra/
  config.sh              nomes, endereço do emulador, forma de chamar o AWS CLI
  01-bucket.sh           bucket diario-graficos, com leitura pública só de graficos/*
  02-tabela.sh           tabela diario-experimentos (projeto + id_experimento)
  03-papel.sh            papel diario-papel-gravar, com permissão mínima
  04-funcao-gravar.sh    empacota app/gravar e publica a função
  deploy.sh              chama os scripts na ordem
  destroy.sh             remove tudo, na ordem inversa
app/gravar/handler.py    a função de gravação
eventos/                 cargas de teste da função
```

Os scripts são bash e rodam no Git Bash (Windows) ou no Terminal (Mac e Linux). Cada um confere se o recurso já existe antes de criar: o `deploy.sh` pode rodar quantas vezes for preciso sem erro e sem apagar dados. O `.gitattributes` força fim de linha LF nos `.sh`, porque o Git do Windows converteria para CRLF e quebraria os scripts.

3. A função de gravação

A função faz, nesta ordem:

1. **Valida tudo antes de escrever qualquer coisa** (RF-04). Recusa com 400, dizendo o campo, se faltar projeto, modelo, métrica, valor ou gráfico; se o valor não for número (como "baixo", "NaN" ou infinito); se o gráfico não for PNG em base64 ou passar de 2 MB.
2. **Normaliza o projeto** para minúsculas, sem espaços nas pontas, para "Previsao-Inflacao" e "previsao-inflacao" serem a mesma partição.
3. **Gera o identificador** — data e hora em UTC até o microssegundo, mais seis caracteres aleatórios — e a data e hora do registro (RF-01).
4. **Grava o gráfico no S3 primeiro**, em `graficos/<projeto>/<id>.png` (RF-02, RNF-03).
5. **Só depois grava o experimento no DynamoDB**, com a condição de que a chave ainda não exista: nada é sobrescrito (RD-02).
6. **Registra no CloudWatch Logs** a data e hora, o projeto e o motivo de cada recusa ou falha, e nunca o conteúdo enviado (RNF-04, RD-01).

Em relação ao código genérico do roteiro, cinco coisas mudaram, cada uma por causa de um documento já entregue:

Código do roteiro	Nosso código	Por quê
Campos genéricos `id_particao`, `descricao`, `valor`	Os campos do RF-01: `projeto`, `modelo`, `hiperparametros`, `metrica`, `valor`, `grafico`	O RF-01 lista exatamente estes campos
Grava no S3 o próprio registro em JSON	Grava no S3 o PNG do gráfico	O RF-02 promete guardar o gráfico do resultado
Valor como texto com duas casas	Valor como número exato (`Decimal`), com vírgula ou ponto	Duas casas transformariam 0,412 em 0,41 e quebrariam o US-01, cenário 1
Identificador aleatório (UUID)	Identificador que começa pela data e hora	A tabela já devolve a lista do mais recente ao mais antigo (seção 6.2 da arquitetura)
Um papel com `"Resource": "*"`	Papel com cada recurso nomeado e sem exclusão	É o que o RNF-02 promete (desafio opcional 1 do roteiro)

4. A política do papel

```
{
  "Statement": [
    { "Sid": "GuardarGrafico",     "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::diario-graficos/graficos/*" },
    { "Sid": "InserirExperimento", "Action": "dynamodb:PutItem",
      "Resource": "arn:aws:dynamodb:us-east-1:000000000000:table/diario-experimentos" },
    { "Sid": "EscreverRegistro",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:us-east-1:000000000000:log-group:/aws/lambda/diario-gravar:*" }
  ]
}
```

Zero ações de exclusão ou de atualização e nenhum `Resource` igual a `*`. O LocalStack gratuito não aplica o IAM, então o requisito é conferido pelo que está escrito na política.

5. Como rodar

```
lstk start
./infra/deploy.sh
source infra/config.sh
$AWS lambda invoke --function-name diario-gravar --payload fileb://eventos/gravar-ok.json saida.json
cat saida.json
$AWS dynamodb scan --table-name diario-experimentos
$AWS logs tail /aws/lambda/diario-gravar --format short
./infra/destroy.sh
```

6. Verificação

O que foi conferido	Resultado
`deploy.sh` a partir do emulador vazio	cria bucket, tabela, papel e função; entre 40 s e 65 s
`deploy.sh` rodado de novo	reaproveita o que existe ("ja existe"), sem erro
Invocação com `gravar-ok.json` (ARIMA, RMSE "0,412")	201, com identificador e endereço do gráfico
O item na tabela	exatamente 1 experimento, `valor` = `0.412` (número, sem arredondar)
O gráfico no S3	252 bytes, e o arquivo baixado pelo endereço devolvido é idêntico ao enviado
Invocação com `gravar-valor-baixo.json`	400, "o valor da métrica precisa ser numérico"; continua 1 item e 1 gráfico
14 casos de borda	10 recusas com 400 e o campo certo (campos ausentes, NaN, infinito, "1.234,5", `true`, PDF, base64 inválido, gráfico acima de 2 MB); valores 0, -0,15 e 0.385 gravados exatos; "Teste-Borda " gravado como `teste-borda`; experimento sem hiperparâmetros aceito
Bucket fora do ar (RNF-03)	503 "não foi possível guardar o gráfico", e nenhum experimento novo na tabela
Política do papel (RNF-02)	0 ações de exclusão ou alteração, 0 recursos `*`
CloudWatch Logs (RNF-04)	uma linha `gravado`, `recusado` ou `falha` por execução, com data e hora, projeto e motivo
`destroy.sh` → `deploy.sh` → invocação	tudo removido, recriado, e a função volta a responder 201

7. A parte que é só da equipe

O roteiro pede três decisões registradas, com a alternativa descartada. Elas estão na seção 9 do `docs/02-arquitetura.md`:

Decisão	Alternativa descartada	Por quê
Nomes dos campos do cenário (`projeto`, `id_experimento`, `criado_em`, `modelo`, `hiperparametros`, `metrica`, `valor`, `grafico`)	Os nomes genéricos do roteiro	Quem abre a tabela ou o log entende sem tradução, e os nomes são os do RF-01
A função valida os cinco campos obrigatórios, o valor como número finito com vírgula ou ponto, e o gráfico como PNG de até 2 MB	Recusar valor zero ou negativo, como o roteiro	Métrica não é preço: um R^2 pode ser negativo, e um erro pode ser zero

Decisão	Alternativa descartada	Por quê
O log leva data e hora, projeto e motivo de cada recusa ou falha, e o identificador de cada gravação	O corpo inteiro da requisição	O corpo traz o gráfico em base64 e textos livres; o RD-01 manda manter conteúdo fora do log

8. Checklist final do roteiro

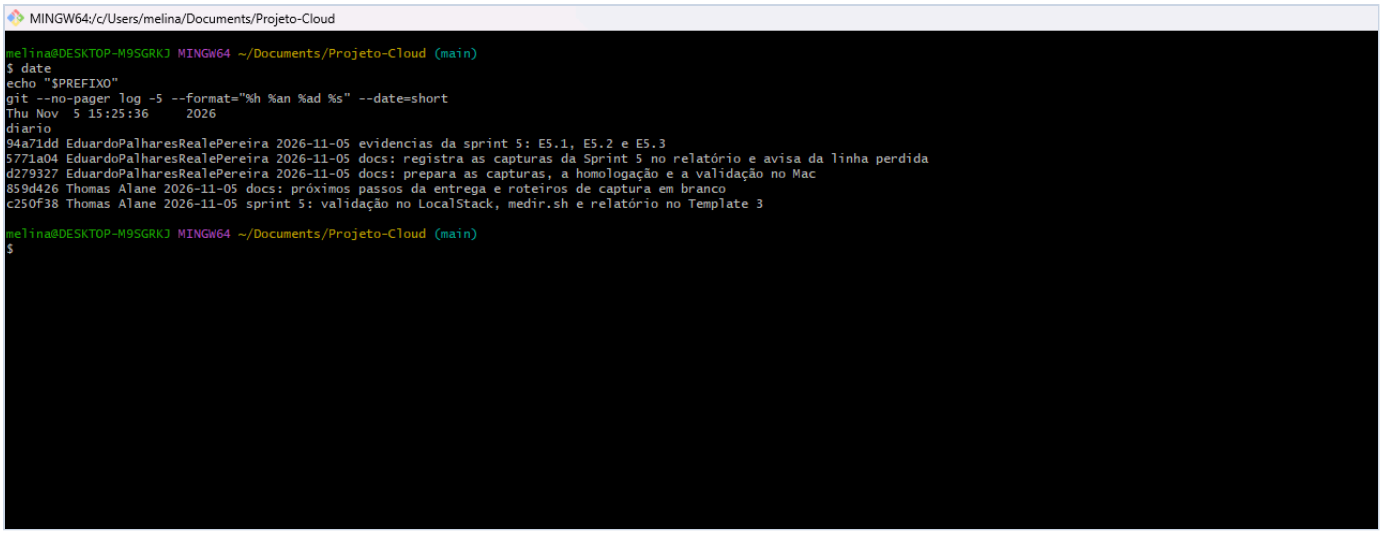
Item	Situação
Criei a estrutura de pastas `infra/`, `app/` e `eventos/`	feito
Todos os nomes de recurso estão no `config.sh`	feito
O bucket é criado por script e o script é idempotente	feito
A tabela usa a chave de partição justificada na Sprint 2	feito: `projeto`
O papel IAM existe e tem política anexada	feito, com recursos nomeados
A função Lambda foi empacotada e publicada por script	feito
A invocação com dado válido devolve 201	feito
O item aparece no DynamoDB	feito
O arquivo aparece no S3	feito: o PNG do gráfico
A linha do `print` aparece no CloudWatch Logs	feito
A invocação com dado inválido devolve 400 e não grava nada	feito
O `deploy.sh` cria tudo em uma execução	feito
O `destroy.sh` remove tudo sem falhar em recurso ausente	feito
Rodei `destroy.sh` e `deploy.sh` em sequência e a função voltou a responder	feito
Nenhum `.zip` foi versionado	feito: `*.zip` no `.gitignore`
Nenhuma credencial foi versionada	feito
Executei `git push`	feito

9. Evidências de execução

Capturas do roteiro de evidências da Sprint 3 (`docs/evidencias/evidencias-sprint3.docx`), feitas no computador de quem rodou, com `date` no início, o prompt visível e o prefixo `diario`.

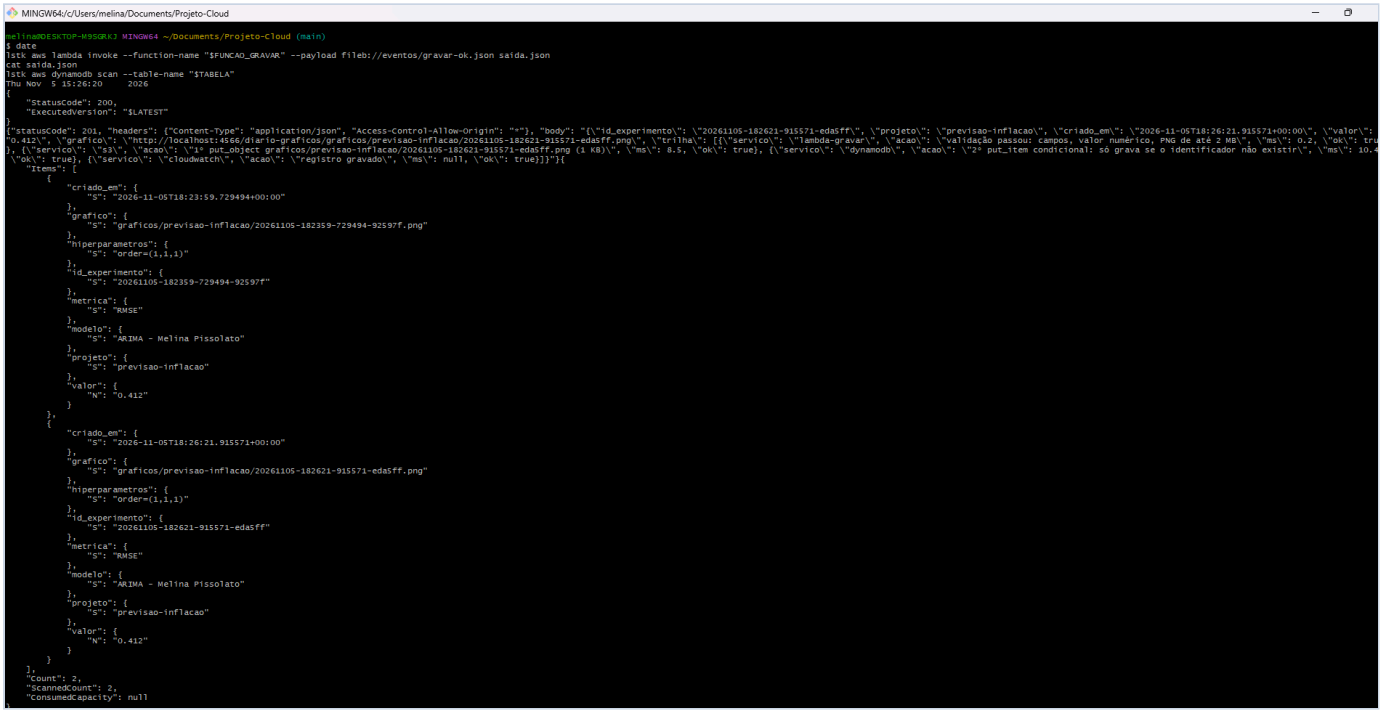
Conferência	Valor da execução
Prefixo da equipe	`diario`
Nome da tabela	`diario-experimentos`
Chave de partição da tabela	`projeto`
Identificador devolvido na E3.2	`20261105-182621-915571-eda5ff`

E3.1 — O projeto e quem fez: prefixo e histórico de commits



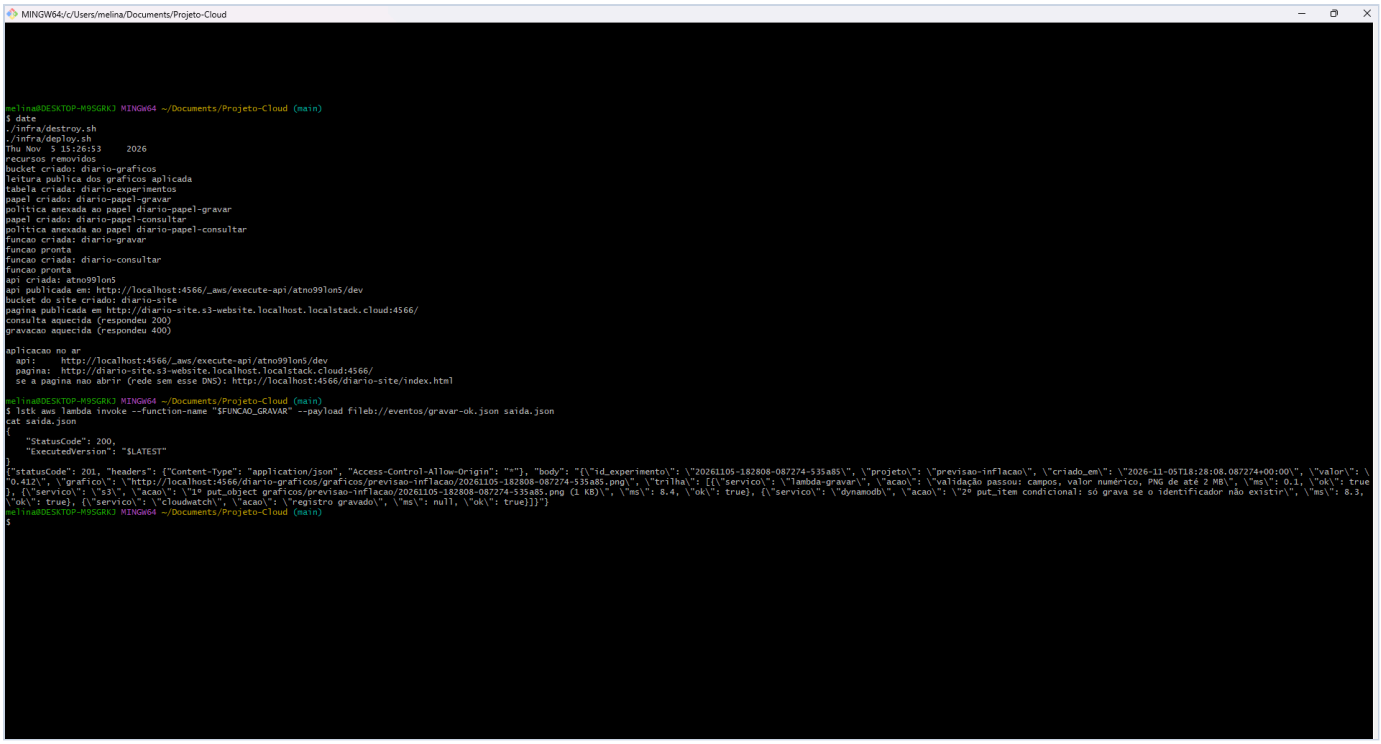
Quem rodou: Melina Pissolato

E3.2 — A função grava (captura de tela 7): 201 e o item na tabela



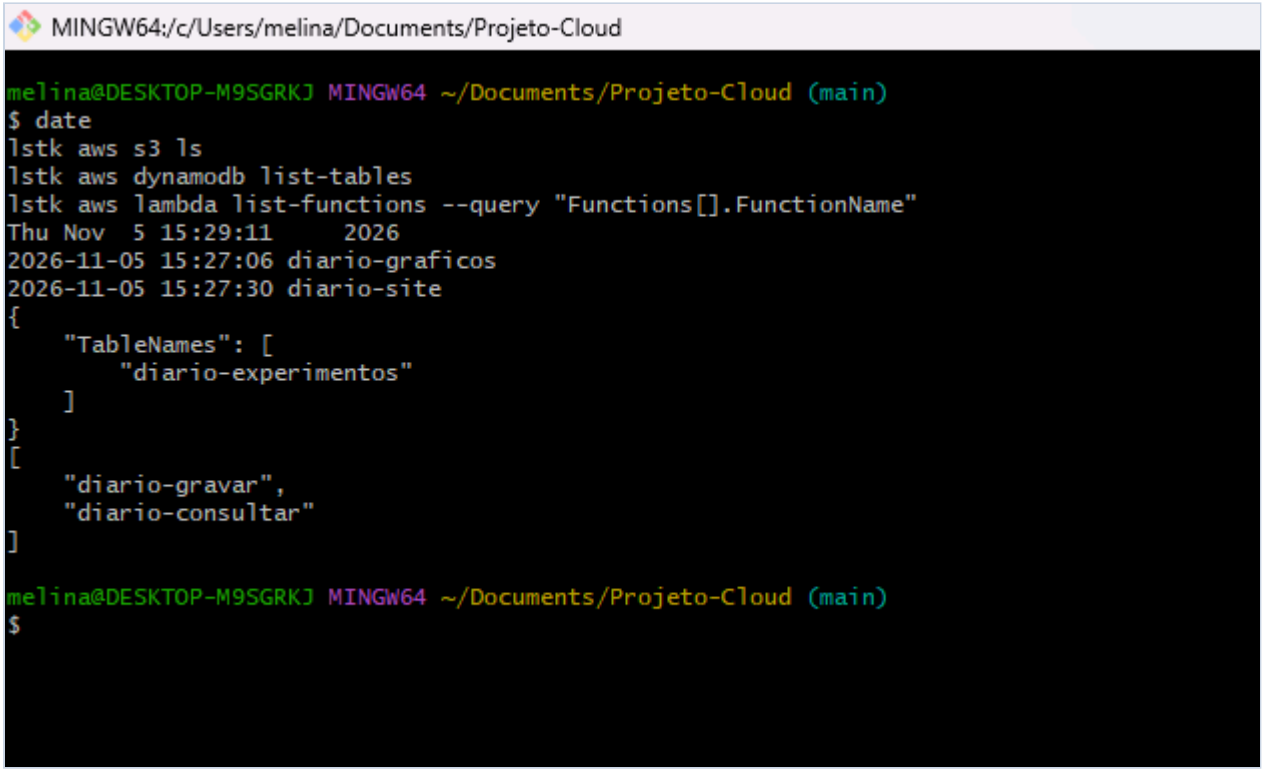
Quem rodou: Melina Pissolato

E3.3 — O ambiente se reconstrói (captura de tela 8): destroy, deploy e a gravação de novo



Quem rodou: Melina Pissolato

E3.4 — Os recursos no LocalStack: bucket, tabela e função



Quem rodou: Melina Pissolato